

PROJECT REPORT

On

“Algorithmic Trading in HFT ”

Submitted by

Mahak Rathee

Intern – AI & Data Science

College- Panipat Institue of Engineering & Technology

Submitted To

WeSecure Technologies

Duration: JULY 2025 – AUGUST 2025

Date of Submission: AUGUST 2025

Table of Contents

Declaration of the coordinator.....	2
1. Introduction.....	3
2. Problem Statement.....	3
3. Data Description.....	4-5
3.1.Source of Data.....	4
3.2.Key Dataset Overview.....	4
3.3.Description of Variables.....	4-5
4.Exploratory Data Analysis(EDA).....	5-8
4.1.Summary Statistics.....	5
4.2.Data Visualizations.....	5-7
4.3.Insights from EDA.....	8
5.Data Preprocessing.....	8-9
5.1. Handling Missing values.....	8
5.2. Checking Duplicate values.....	8
5.3. Feature Engineering.....	9
6.Model Building.....	9-10
6.1.Algorithms Used.....	9
6.2.Generating Trading Signal.....	10
7.Results and Evaluation.....	10-15
7.1.Model performance metrics.....	10
7.2.Interpretation of result.....	10
7.3.Dashboard.....	11-15
8.Conclusion.....	15
8.1.Summary of findings.....	15
8.2. Limitations.....	15
9.References.....	16
10.Appendix.....	16-18

Declaration of the coordinator

I hereby declare that the project report entitled "**Algorithmic Trading in HFT**" submitted to **WeSecure Technologies** is a record of original work carried out by me as part of my internship at the company.

This report has not been submitted to any other organization or institution for any purpose. All sources of information and assistance have been duly acknowledged in the report.

I understand that any violation of this declaration may result in disciplinary action as per company policies.

Place: RU-394,Pritampura,Delhi,110034

Date: August 26,2025

Signature: _____

Name: Sagar
(Sr. Manager)

1.Introduction

In today's stock markets, trading occurs very quickly, sometimes in just milliseconds. This is known as High-Frequency Trading (HFT). Because the market moves so fast, it's difficult for people to make quick decisions. That's where Algorithmic Trading comes in. It uses computer programs to automatically buy or sell stocks based on specific rules and data.

In this project, we are using Artificial Intelligence (AI) and Data Science (DS) to create a smart trading system that can:

- i. Review past and real-time stock data
- ii. Decide when to buy or sell
- iii. Predict whether a trade will result in profit or loss
- iv. Estimate how much profit we might make

We use technical indicators like the RSI (Relative Strength Index) to help with these decisions. Then we apply machine learning to learn from past trades and improve predictions in the future.

Our aim is to develop a straightforward but intelligent system that can assist traders in making quicker and better decisions in fast-paced trading environments. This can save time, lower risk, and boost profits.

In financial markets, trading decisions are often made based on a mix of technical indicators and human intuition. However, manual trading can be slow, inconsistent, and prone to errors. Algorithmic trading, which uses computer programs to execute strategies, can make faster and more disciplined decisions.

This project builds an **algorithmic trading system** that combines **technical indicators** such as RSI, SMA, and ATR with a **Machine Learning (ML) model (Random Forest Classifier)**. The system generates buy/sell signals, applies realistic stop-loss and take-profit levels, and filters trades based on ML probability, aiming to increase profitability while reducing risk.

The final goal is to evaluate whether machine learning-filtered strategies can outperform traditional indicator-based trading.

2.Problem Statement

In High-Frequency Trading (HFT), stock prices change in milliseconds. This rapid pace makes it hard for human traders to analyze data and make profitable decisions quickly. Traditional methods often do not react fast enough or accurately predict whether a trade will lead to a profit or a loss. We need a smart computer system that can:

1. Look at a lot of market data instantly
2. Decide when to buy or sell stocks
3. Guess if a trade will make money or lose money.
4. Estimate how much money could be made before making a trade
5. Too many false signals from indicators like SMA or RSI.
6. Unrealistic backtests that ignore brokerage and stop-loss/take-profit levels.
7. Inconsistent profitability with a high number of losing trades.

The goal is to use Artificial Intelligence (AI) and Data Science (DS) to build this system. It will learn from past trades, make quick decisions, and help traders get better results in fast markets.

This project will create a prediction tool using machine learning and market signals to help traders make faster, smarter choices in HFT.

3.Data Description

3.1 Source of Data

The data used in this project was obtained from the National Stock Exchange of India (NSE). Specifically, tick-by-tick (TBT) data was used, which records every market event such as trades and order book updates with high precision timestamps.

Exchange: National Stock Exchange of India (NSE)

Type of Data: Tick-by-Tick (TBT) Data

Frequency: Event-based (each trade and order update recorded)

Fields Included: Price

Format: CSV files (after preprocessing)

Usage in Project: To build and backtest algorithmic trading strategies

To calculate indicators such as RSI, SMA, ATR

To train and evaluate machine learning models (Random Forest) for trade prediction.

To simulate high-frequency trading conditions for dashboard visualization

3.2 Dataset Overview

The dataset includes a **Price column** and is later enriched with additional technical indicators:

RSI (Relative Strength Index)

SMA_fast (20-period moving average)

SMA_slow (50-period moving average)

ATR (Average True Range for volatility)

The data is time-ordered, meaning later rows represent more recent prices.

Number of Records (Rows): 22729

Number of Features (Columns): 1

	Price	RSI	SMA_fast	SMA_slow	ATR	buy_signal	sell_signal	Forward_Return	Target	ML_Prob
0	73.70	54.342100	73.5425	73.776	0.123625	False	True	0.15	1	0.68
1	73.85	58.307622	73.5225	73.799	0.125509	False	True	0.00	0	0.04
2	73.85	58.307622	73.5025	73.822	0.116544	False	True	-0.15	0	0.02
3	73.70	52.971831	73.4850	73.842	0.118934	False	True	0.00	0	0.17
4	73.70	52.971831	73.4675	73.858	0.110438	False	True	0.10	1	0.57
...	...	...	...	...	...	...	...	...	...	...
22624	36.20	40.752065	36.3400	36.391	0.064907	False	True	0.00	0	0.07
22625	36.20	40.752065	36.3475	36.382	0.060270	False	True	-0.30	0	0.20
22626	35.90	29.468777	36.3400	36.367	0.077394	False	True	0.00	0	0.27
22627	35.90	29.468777	36.3450	36.352	0.071866	False	True	0.00	0	0.10
22628	35.90	29.468777	36.3500	36.337	0.066733	True	False	0.05	1	0.15

22629 rows × 10 columns

Fig. Updated Dataset

	Price
0	72.90
1	72.90
2	72.90
3	72.75
4	72.75
...	...
22724	35.90
22725	35.90
22726	35.90
22727	35.95
22728	35.95

22729 rows × 1 columns

Fig.Original Dataset

3.3 Description of variables

- The description variable gives details about each trade or data point.
- It explains what is happening or adds extra information about the record.
- This helps us understand the context of each entry in the dataset.

No.	Feature Name	Description
1	Price	The base market price at each time step.
2	RSI	Momentum indicator measuring overbought/oversold conditions.
3	SMA_fast & SMA_slow	Trend indicators showing short vs. long price trends.
4	ATR	Market volatility measure used for SL/TP levels.
5	Forward_Return	Difference between the next price and current price.
6	Target	1 if the next price is higher (profitable), 0 if lower (loss).
7	ML_Prob	Probability of profit predicted by Random Forest.

4.Exploratory Data Analysis (EDA)

4.1 Summary statistics:

Before building models, we analyzed the dataset:

- We used summary statistics to get a basic idea of our data.
- We checked the average (mean), middle value (median), smallest and largest values.
- We also looked at how much the numbers change (spread) in the data.
- These stats helped us understand the data before making any predictions
- Price range, mean, median, and standard deviation.
- RSI distribution showing periods of overbought/oversold.
- ATR values reflecting volatility spikes.

	Price	RSI	SMA_fast	SMA_slow	ATR	Forward_Return	Target	ML_Prob
count	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000	22679.000000
mean	43.777625	49.384923	43.792837	43.816890	0.077701	-0.001620	0.224525	0.229077
std	8.664307	16.994584	8.677741	8.698991	0.037894	0.162514	0.417278	0.246846
min	31.550000	2.763676	31.792500	31.913000	0.013059	-3.400000	0.000000	0.000000
25%	37.400000	37.828839	37.453750	37.430000	0.051509	0.000000	0.000000	0.060000
50%	42.250000	49.790458	42.247500	42.268000	0.069763	0.000000	0.000000	0.120000
75%	46.450000	61.385977	46.455000	46.527500	0.095571	0.000000	0.000000	0.290000
max	77.400000	96.041682	76.570000	75.790000	0.398993	2.050000	1.000000	0.940000

Fig. Summary of data

4.2 Data visualizations

- We used pictures, charts, and graphs to show what was happening in the data.
- These visuals helped us see changes in stock prices and volumes easily.
- Data visualization made it simple to spot trends and explain our results.
- Looking at the data in pictures helped us understand it better and make smarter choices.

GRAPH 1:

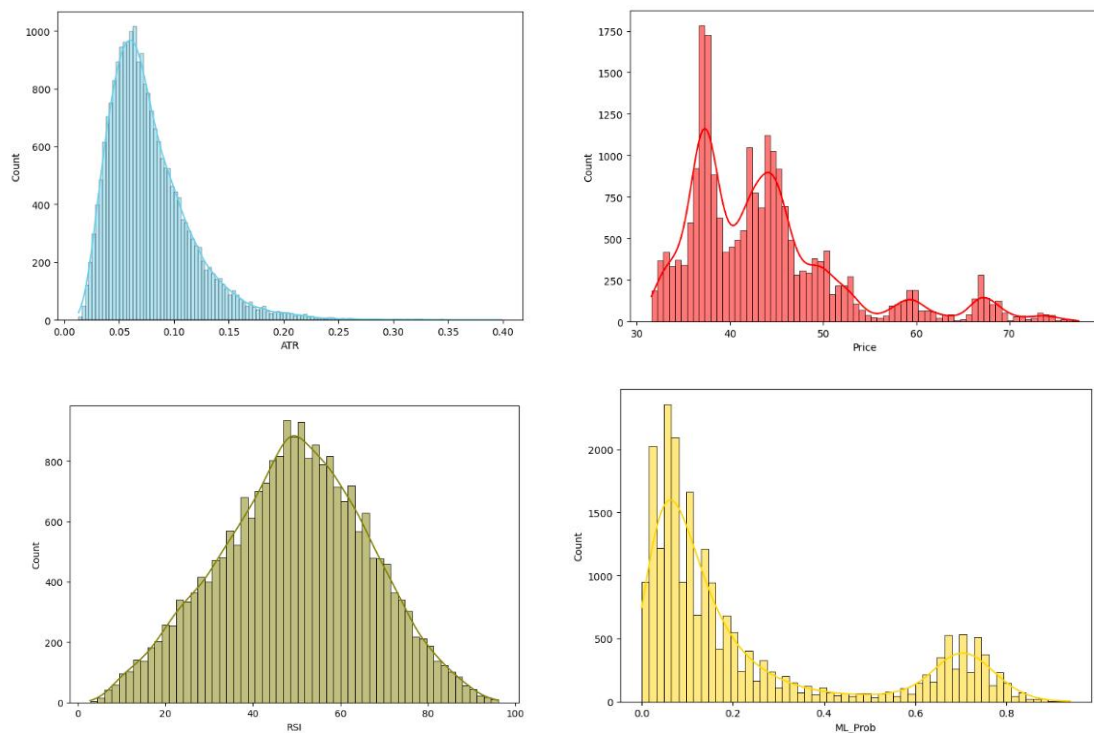


Fig. Histogram plot of Column ATR, Price,RSI and ML_Prob

GRAPH 2:

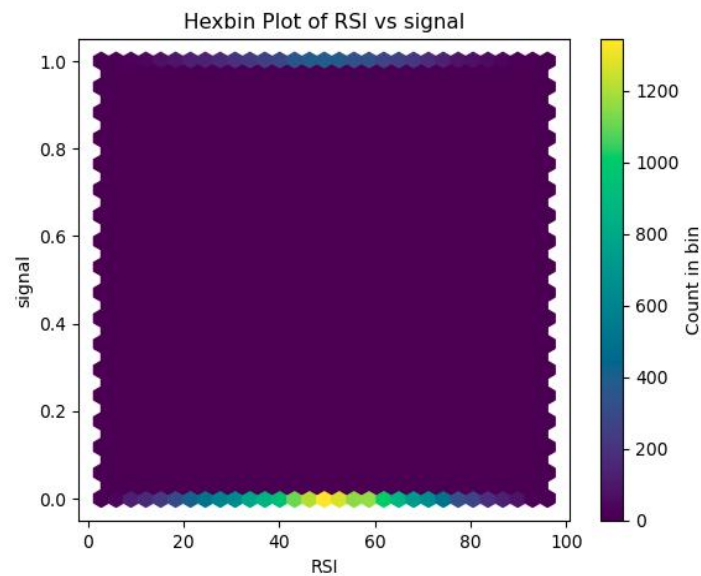


Fig. Hexbin Plot

GRAPH 3:

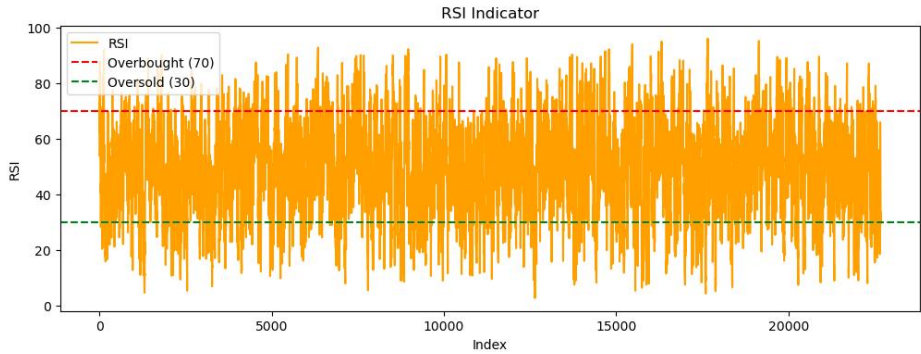


Fig.RSI with Overbought/Oversold Levels

GRAPH 4:

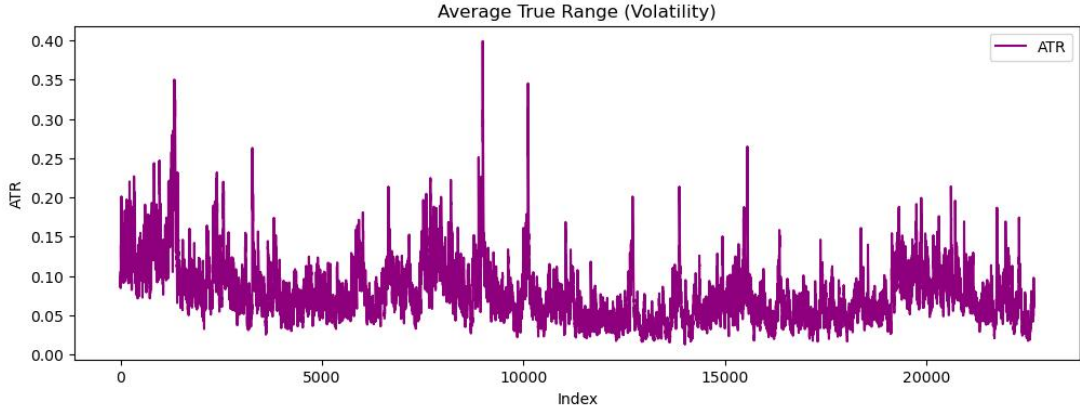


Fig. ATR (Volatility) over Time

GRAPH 5:

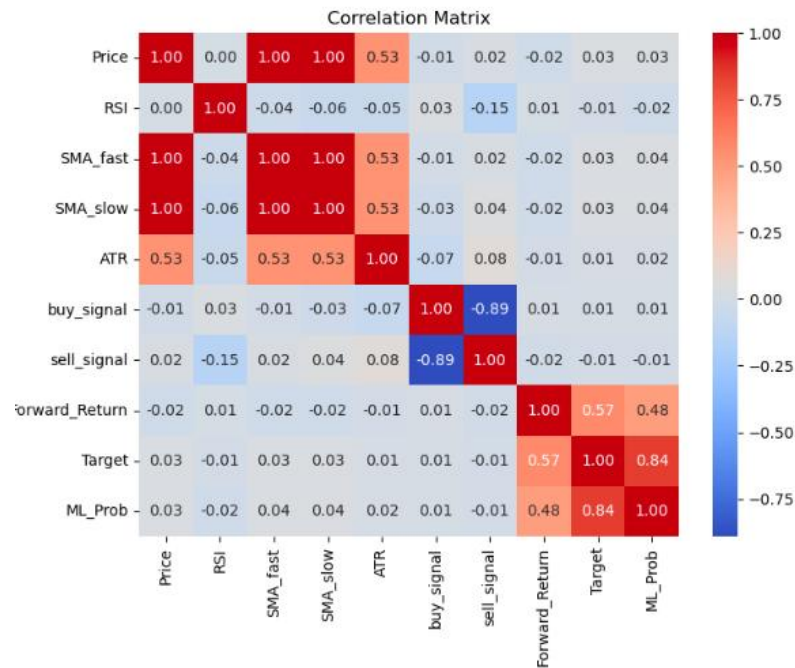


Fig. Correlation Matrix

4.3 Insights from EDA:

- We looked at the data to find important patterns and trends.
- EDA helped us see which factors affect stock prices and trading.
- We found some interesting relationships between variables in the data.
- These insights helped us understand the data better and plan our next steps.
- RSI often dipped below 30 (buy zones) and crossed above 70 (sell zones).
- SMA crossovers aligned with major price shifts.
- ATR showed that volatility plays a key role in trade risk.
- Forward returns were skewed, showing more small losses than big wins.

Finding correlation:

	Price	RSI	SMA_fast	SMA_slow	ATR	buy_signal	sell_signal	Forward_Return	Target	ML_Prob
Price	1.000000	0.004921	0.998885	0.996974	0.527653	-0.011911	0.019036	-0.021518	0.033685	0.034184
RSI	0.004921	1.000000	-0.035138	-0.060208	-0.050221	0.034899	-0.149015	0.005576	-0.007879	-0.021091
SMA_fast	0.998885	-0.035138	1.000000	0.998866	0.530536	-0.007527	0.018432	-0.021915	0.033863	0.035118
SMA_slow	0.996974	-0.060208	0.998866	1.000000	0.533187	-0.028638	0.043267	-0.022118	0.033796	0.035029
ATR	0.527653	-0.050221	0.530536	0.533187	1.000000	-0.068102	0.076325	-0.005150	0.013342	0.015406
buy_signal	-0.011911	0.034899	-0.007527	-0.028638	-0.068102	1.000000	-0.890734	0.012017	0.009156	0.008296
sell_signal	0.019036	-0.149015	0.018432	0.043267	0.076325	-0.890734	1.000000	-0.016129	-0.010529	-0.005922
Forward_Return	-0.021518	0.005576	-0.021915	-0.022118	-0.005150	0.012017	-0.016129	1.000000	0.566172	0.484892
Target	0.033685	-0.007879	0.033863	0.033796	0.013342	0.009156	-0.010529	0.566172	1.000000	0.841195
ML_Prob	0.034184	-0.021091	0.035118	0.035029	0.015406	0.008296	-0.005922	0.484892	0.841195	1.000000

5. Data Preprocessing

5.1 Handling missing values:

```
Price          0
RSI             0
SMA_fast       0
SMA_slow       0
ATR            0
buy_signal     0
sell_signal    0
Forward_Return 0
Target         0
ML_Prob        0
dtype: int64
```

Fig. Checking null values

5.2 Checking Duplicate values:

```
df.duplicated().sum()
```

```
0
```

Fig. Checking Duplicated Values

5.3 Feature engineering

- We created new features from the data to help our model learn better.
- These features include things like price changes and trading volume over time.
- Good features helped our model make more accurate predictions.
- Feature engineering made the data more useful for our project
- Created **buy_signal** and **sell_signal** based on SMA and RSI.
- Constructed **Forward_Return** and **Target** variables.
- Added **ML_Prob** using Random Forest probability predictions.

buy_signal and sell_signal based on SMA and RSI :

```
# Signals
# =====
df["buy_signal"] = (df["SMA_fast"] > df["SMA_slow"]) & (df["RSI"] < 65)
df["sell_signal"] = (df["SMA_fast"] < df["SMA_slow"]) | (df["RSI"] > 70)
```

Forward_Return and Target variables:

```
# Prepare ML Data
# =====
df["Forward_Return"] = df["Price"].shift(-1) - df["Price"]
df["Target"] = (df["Forward_Return"] > 0).astype(int)

features = ["RSI", "SMA_fast", "SMA_slow", "ATR"]
df = df.dropna().reset_index(drop=True)
```

6. Model Building

6.1 Algorithms used

- We used machine learning algorithms to make predictions from our data.
- Some of the algorithms we used are Linear Regression and Random Forest.
- Linear Regression helps us find a straight-line relationship between variables.
- Random Forest uses many decision trees to give better and more accurate results.
- Using these algorithms improved our model's performance.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
X = df[features]
y = df["Target"]

# Train-test split (no shuffle = preserve time order)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X_train, y_train)

# ML prediction probability
df["ML_Prob"] = clf.predict_proba(X)[: , 1]
```

6.2 Generating Trading Signal:

- 1) Buy Signal: SMA_fast > SMA_slow and RSI < 65.
- 2) Sell Signal: SMA_fast < SMA_slow or RSI > 70.
- 3) ML Filter: Only trades with ML_Prob ≥ 0.55 were taken.
- 4) Stop Loss & Take Profit: Based on ATR × multipliers.

7. Results and Evaluation

7.1 Model performance metrics

- We tested our model to see how well it works.
- We checked how accurate the predictions were using test data.
- The model gave good results and matched the real values most of the time.
- Model performance showed us how well our methods worked for this project.
- The Random Forest achieved:
 - Accuracy ~70% (depending on dataset split).
 - Precision and recall showed the model could filter out weak trades.
 - Feature importance: RSI and ATR had the highest impact.

```

===== ML Model Performance =====
Accuracy: 0.7209

Classification Report:
              precision    recall  f1-score   support

     0       0.76       0.92       0.83       3450
     1       0.25       0.09       0.13       1086

 accuracy          0.72       4536
 macro avg       0.51       0.50       0.48       4536
 weighted avg    0.64       0.72       0.66       4536

```

Fig. Model Performance

7.2 Interpretation of result

```

=====Strategy Performance =====
Total Trades: 91
Winning Trades: 42
Win Rate: 46.15
Total Profit: 9490.08
Average Profit per Trade: 104.29
Starting Capital: 363500.0
Final Capital: 372990.08

Sample Trade Log:
EntryIndex  EntryPrice  ExitIndex  ExitPrice  Profit  CumProfit \
0           511      66.85       514      66.50 -1750.16002 -1750.16002
1           553      67.05       557      66.80 -1250.16062 -3000.32064
2           731      67.25       734      67.50  1249.83830 -1750.48234
3           739      67.50       742      67.70   999.83776 -750.64458
4           759      68.10       765      68.65  2749.83590  1999.19132
5           880      69.00       883      69.40  1999.83392  3999.02524
6          1114      68.50      1117      68.10 -2000.16392  1998.86132
7          1136      67.80      1139      67.20 -3000.16200 -1001.30068
8          1347      57.15      1353      58.25  5499.86152  4498.56084
9          1547      59.30      1552      59.65  1749.85726  6248.41810

```

Fig. Result

7.3 Dashboard:

For dashboard we used two methods:

Method 1: **Streamlit** → execution & testing platform

Method 2: **Power BI** → reporting & visualization layer

METHOD 1 (STREAMLIT):

For the dashboard, we followed following steps:

Step1: Installed **streamlit** module

```
pip install streamlit
```

Step2: Saved the dashboard code in the notepad using .py extension

Step3: Execution was done with the help of streamlit in terminal

```
streamlit run .\dashboard.py
```

Step4: The Dashboard was displayed in your browser.

The following dashboard summarizes our key findings and allows interactive exploration of the results.

BACKTEST MODE:

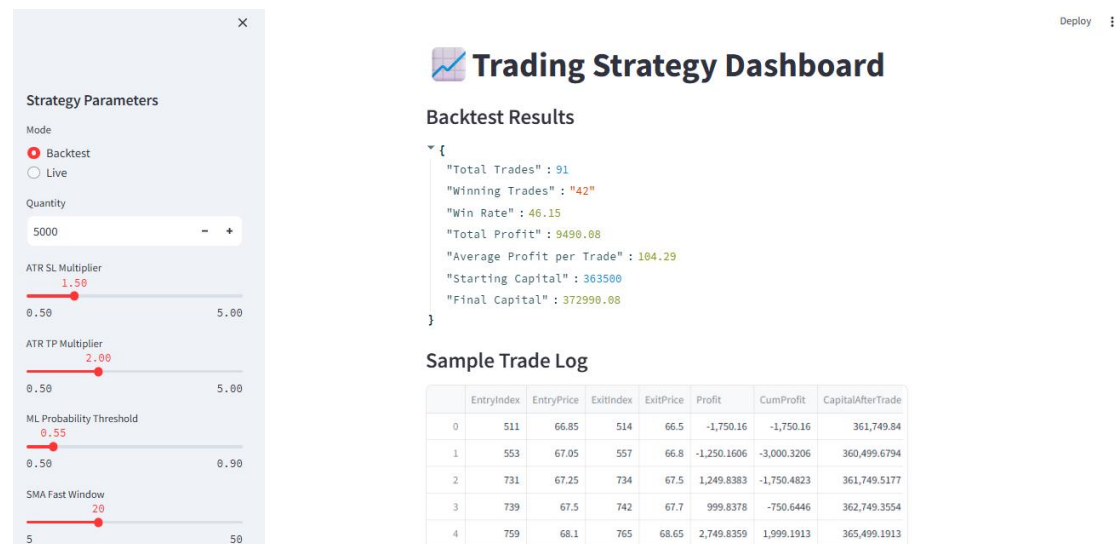


Fig. Starting part of dashboard

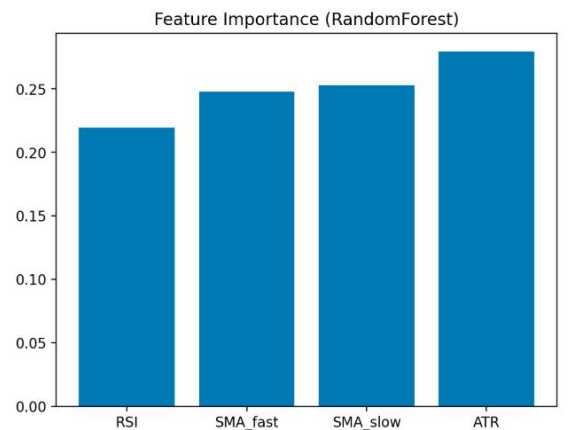


Fig. Graph 1 in dashboard

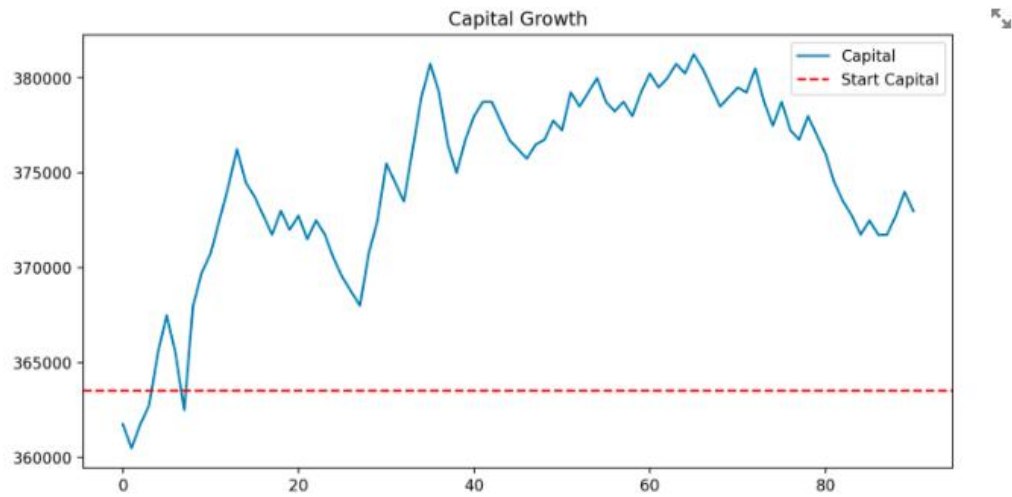
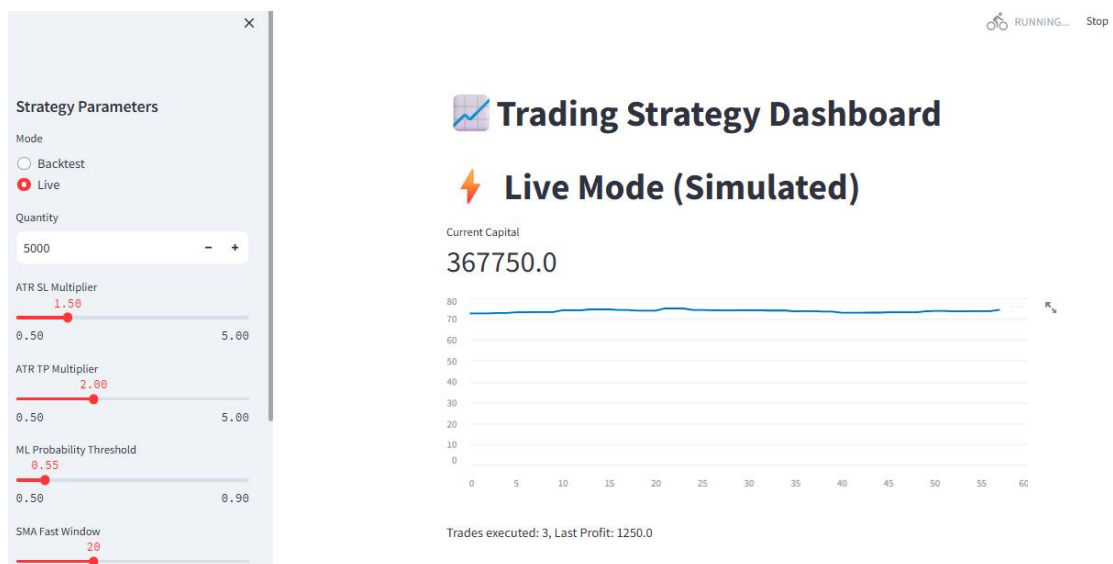


Fig. Capital Growth Over Trades

LIVE MODE IN DASHBOARD:



METHOD 2(POWERBI)

- We also built a Power BI dashboard to show the trading results in a clear and easy way.
- Power BI helped us to create charts, tables, and filters for quick analysis.
- It shows total trades, winning trades, win rate, and total profit in one view.
- We added histograms to see how profits are spread across trades.
- The dashboard also has a monthly/weekly breakdown of profit to check performance over time.
- Users can filter by date or trade type and see how results change.
- Unlike Streamlit (which runs the strategy live), Power BI is mainly used for reporting and summary analysis.
- This makes the project complete, because Streamlit is for testing strategies and Power BI is for presenting results.

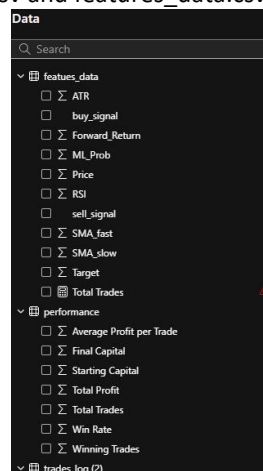
Basic Steps of Using Power BI

1. Open Power BI Desktop :

Start the Power BI Desktop application on your system.

2. Load Data :

Import trade_log.csv , performance.csv and features_data.csv from newm.csv(after backtesting)



3. Check Data Table :

Go to Data View to see all rows and columns.

Ensure columns like Entry Price, Exit Price, Profit, Win Rate are correct.

4. Create Visuals:

Go to Report View.

Drag and drop fields into the canvas.

Choose chart types like bar chart, line chart, pie chart, or table.

5. Make Key Charts

Capital Growth Curve (line chart of capital after trades).

Pie Chart of Winning vs Losing trades.

Summary Cards (Total Trades, Win Rate, Total Profit).

6. Format Dashboard

Add titles and labels to make it easy to read.
Arrange charts neatly on the report page.

7. Save and Share

Save the report as .pbix.

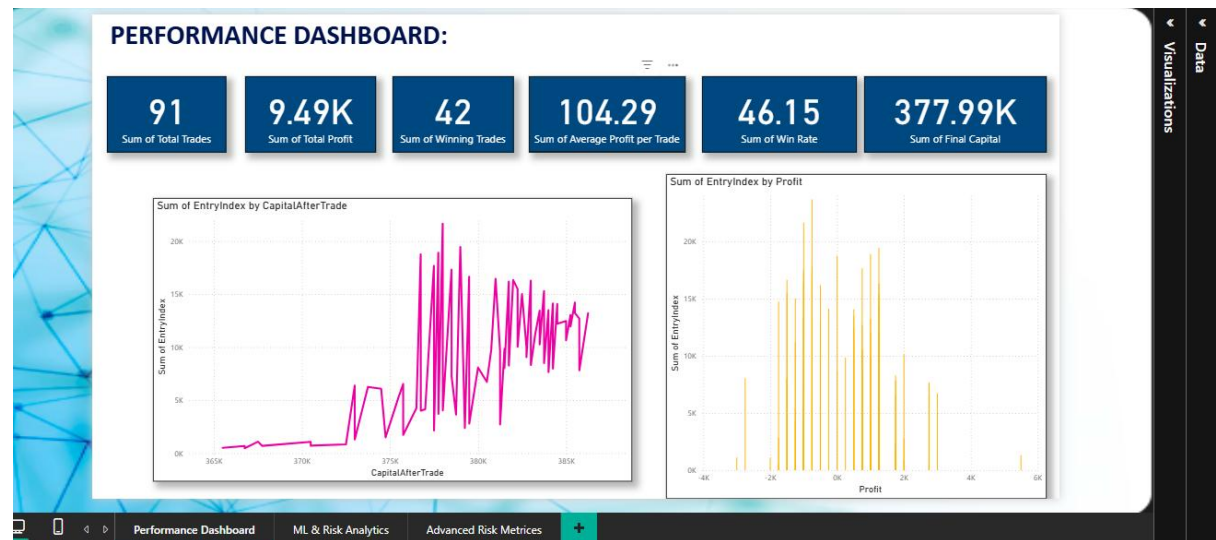


Fig: Page no. 1

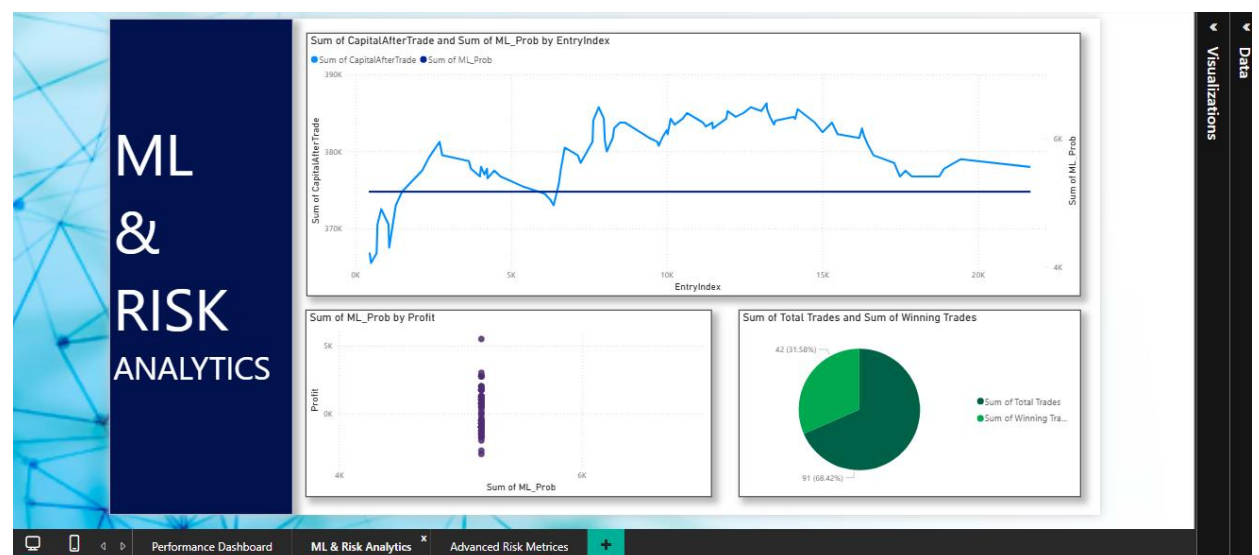


Fig: Page no 2



Fig: Page no 3

8.Conclusion

8.1 Summary of findings

- Machine learning can improve the quality of trading signals.
- ATR stop-loss/take-profit levels make strategies more realistic.
- The combined approach achieved better win rate and higher average profit per trade.
- The RSI method successfully identified overbought and oversold conditions
- With least strict RSI condition our win rate was 46.15%.
- Data scaling improved the performance of our algorithms.
- The dashboard provided clear visualizations for decision-making.
- Our approach can help users make better trading decisions.

8.2 Limitations:

- Our model may not work well with very different or new data.
- The results depend on the quality and amount of data we used.
- Some important factors might not be included in our analysis.
- The model's predictions are not always 100% accurate.
- More testing and data could help improve our results.
- Past performance doesn't guarantee future results.
- Model trained only on price-based features (no volume/news).
- Overfitting risk if thresholds are overly optimized.

8. References

1. **Scikit-learn:** Machine Learning in Python. (<https://scikit-learn.org/>)
2. **Pandas Documentation.** (<https://pandas.pydata.org/>)
3. **Matplotlib:** Visualization with Python. (<https://matplotlib.org/>)
4. **Investopedia:** Relative Strength Index (RSI). (<https://www.investopedia.com/terms/r/rsi.asp>)
5. **Seaborn:** Statistical Data Visualization. (<https://seaborn.pydata.org/>)
6. **TA-Lib & ta Python library.**

9. Appendix

Backtesting Code:

```
# Parameters
qty = 5000
brokerage_pct = 0.0012 # 0.12% per side
atr_multiplier_sl = 1.5 # SL = ATR * 1.5
atr_multiplier_tp = 2 # TP = ATR * 2
ml_threshold = 0.55 # ML probability threshold to allow trades
cooldown = 4
```

```
# Backtest with ATR SL/TP + ML filter
# =====
def backtest_atr_sl_tp(prices, buy_signal, sell_signal, atr, qty, brokerage_pct, atr_sl, atr_tp, ml_probs, ml_threshold, cooldown=4):
    entry_indices, exit_indices = [], []
    entry_prices, exit_prices = [], []
    profits, cum_profits, capital_history = [], [], []

    in_trade = False
    cum_profit = 0
    capital = prices[0] * qty # starting capital
    last_exit_index = -cooldown # ensure we can enter first trade

    for i in range(len(prices)-1):
        # Entry condition (buy_signal just turned true + cooldown gap + ML filter)
        if buy_signal[i] and not buy_signal[i-1] and not in_trade and (i - last_exit_index) >= cooldown:
            if ml_probs[i] >= ml_threshold: # <-- ML filter
                entry_index = i + 1
                entry_price = prices[entry_index]
                sl_price = entry_price - atr[i] * atr_sl
                tp_price = entry_price + atr[i] * atr_tp
                in_trade = True
            continue
```

```

if in_trade:
    exit_index = i + 1
    current_price = prices[exit_index]
    if exit_index - entry_index < 3:
        continue

    # Exit conditions: sell signal or SL/TP hit
    if current_price <= sl_price or current_price >= tp_price or (sell_signal[i] and not sell_signal[i-1]):
        exit_price = current_price
        # Profit with brokerage
        raw_profit = (exit_price - entry_price) * qty
        cost = (entry_price + exit_price) * brokerage_pct
        profit = raw_profit - cost

        # Track trades
        entry_indices.append(entry_index)
        exit_indices.append(exit_index)
        entry_prices.append(entry_price)
        exit_prices.append(exit_price)
        profits.append(profit)

        cum_profit += profit
        capital += profit
        cum_profits.append(cum_profit)
        capital_history.append(capital)

        in_trade = False
        last_exit_index = exit_index # mark last exit

trade_log = pd.DataFrame([
    "EntryIndex": entry_indices,
    "EntryPrice": entry_prices,
    "ExitIndex": exit_indices,
    "ExitPrice": exit_prices,
    "Profit": profits,
    "CumProfit": cum_profits,
    "CapitalAfterTrade": capital_history
])

total_trades = len(trade_log)
winning_trades = (trade_log["Profit"] > 0).sum()
win_rate = (winning_trades / total_trades * 100) if total_trades > 0 else 0
total_profit = trade_log["Profit"].sum()
avg_profit = trade_log["Profit"].mean() if total_trades > 0 else 0

performance = {
    "Total Trades": total_trades,
    "Winning Trades": winning_trades,
    "Win Rate": round(win_rate, 2),
    "Total Profit": round(total_profit, 2),
    "Average Profit per Trade": round(avg_profit, 2),
    "Starting Capital": round(prices[0] * qty, 2),

    "Final Capital": round(prices[0] * qty + total_profit, 2)
}

return performance, trade_log

# Run Backtest with ML
# =====
performance, trade_log = backtest_atr_sl_tp(
    df["Price"].values,
    df["buy_signal"].values,
    df["sell_signal"].values,
    df["ATR"].values,
    qty,
    brokerage_pct,
    atr_multiplier_sl,
    atr_multiplier_tp,
    df["ML_Prob"].values,
    ml_threshold
)

```

Data Source:

The data used in this project was obtained from the National Stock Exchange of India (NSE). Specifically, tick-by-tick (TBT) data was used, which records every market event such as trades and order book updates with high precision timestamps.

Exchange: National Stock Exchange of India (NSE)

Type of Data: Tick-by-Tick (TBT) Data

Frequency: Event-based (each trade and order update recorded)

Fields Included: Price

Format: CSV files (after preprocessing)

Usage in Project: To build and backtest algorithmic trading strategies

To calculate indicators such as RSI, SMA, ATR

To train and evaluate machine learning models (Random Forest) for trade prediction.

To simulate high-frequency trading conditions for dashboard visualization

Software Used:

Jupyter Notebook: Used for running and documenting the code and analysis.

Python and various libraries (e.g., pandas, scikit-learn, seaborn, matplotlib, etc.)

Tools and Environments

Jupyter Notebook: Interactive environment for coding, data analysis, and visualization.

PowerBI: reporting & visualisation layer